


Canny



Edge Detector

Adrienne Macazar



Fondamenti dell'Intelligenza Artificiale 2025-26

Università di Parma

adriennechloerayos.maczar@studenti.unipr.it

Edge Detection

Cos'è un edge?

- Una **discontinuità** nell'intensità luminosa dei pixel, spesso corrispondente al contorno di un oggetto o a un cambio di materiale. Viene individuato tramite **il calcolo dei gradienti**





Tre criteri fondamentali

1

Rilevamento

Massimizzare il **rilevamento degli edge reali** e minimizzare la sensibilità al rumore per **non generare edge falsi**

2

Localizzazione

Gli edge rilevati devono essere **posizionati con precisione** il più vicino possibile al centro di un edge reale

3

Unicità

Quando ci sono due risposte per lo stesso edge, **una dovrà per forza essere falsa**



Canny Edge Detector - Pipeline

00

Preprocessing

Conversione dell'immagine digitale in **scala di grigi**

01

Smoothing

Convoluzione con un **filtro Gaussiano** per ridurre il rumore

02

Gradienti

Calcolo del **magnitudo** e della **direzione** dei gradienti con **filtri di Sobel**

03

NMS

Soppressione dei non massimi lungo la direzione dei gradienti

04

Double Threshold

Classificazione dei pixel in edge forti, deboli o falsi in base a due soglie

05

Isteresi

Promozione di edge deboli connessi a edge forti e **scarto** dei rimanenti isolati

Smoothing Gaussiano

Viene applicata una **convoluzione** con un **kernel Gaussiano 2D** per ridurre il rumore dell'immagine:

- $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$
- La **dimensione del kernel** k^* determina quanti pixel vengono considerati nella convoluzione
- σ^* è la **deviazione standard** che controlla l'intensità della sfocatura

```
def gaussian_kernel(size=5, sigma=1.0):  
    """  
    Creates a 2D Gaussian kernel matrix for the Gaussian blur step.  
    size: kernel size, use odd numbers (3, 5, 7...).  
    sigma: standard deviation, controls the blur strength.  
    """  
    # Calculate the distance from the kernel center.  
    kernel_center = size // 2  
    # Create a 1D array of integer offsets from the center:  
    #   e.g size=5 -> [-2, -1, 0, 1, 2]  
    offset = np.arange(-kernel_center, kernel_center + 1)  
  
    # Apply 1D Gaussian to the offset.  
    gauss_1d = np.exp(-offset**2 / (2 * sigma**2))  
  
    # Produce the 2D Gaussian kernel from two 1D Gaussians.  
    # np.outer -> k[i][j] = gauss_1d[i] * gauss_1d[j]  
    kernel = np.outer(gauss_1d, gauss_1d)  
  
    # Return the normalized kernel.  
    # array.sum adds all kernel values.  
    # Divide the kernel by the value so that all kernel values sum to 1.  
    return kernel / kernel.sum()
```

* k e σ sono parametri scelti dall'utente; i loro valori di default sono 5 e 1.0

Calcolo dei Gradienti

Vengono calcolate le **derivate prime** nelle direzioni orizzontale e verticale tramite i **filtri di Sobel**:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Dalla convoluzione ricaviamo **magnitudo** e **direzione** di ogni edge:

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan2(G_y, G_x)$$

```
def sobel_gradients(img):
    Kx = np.array([[ -1,  0,  1],
                   [-2,  0,  2],
                   [-1,  0,  1]], dtype=np.float64)

    Ky = np.array([[ -1, -2, -1],
                   [ 0,  0,  0],
                   [ 1,  2,  1]], dtype=np.float64)

    Gx = img_convolve(img, Kx)
    Gy = img_convolve(img, Ky)

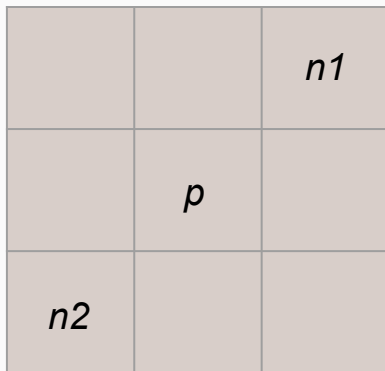
    magnitude = np.sqrt(Gx**2 + Gy**2).
    img_edges = (magnitude / magnitude.max()) * 255
    angles = np.arctan2(Gy, Gx) * (180 / np.pi)

    # Clip the angles to have them in range [0°, 180°].
    # Use numpy's boolean indexing:
    #   angle < 0 creates a boolean array.
    #   += 180 adds 180 where the condition is True.
    angles[angles < 0] += 180

    return img_edges.astype(np.uint8), angles
```

Non-Maximum Suppression

Ogni pixel viene mantenuto soltanto se è il **massimo locale lungo la direzione del gradiente θ** , altrimenti viene azzerato. In questo modo si ottengono **edge sottili**.



135°

```
def nms(sobel_edges, angles):  
    """  
    Performs non-maximum suppression on each pixel to thin edges by  
    keeping only the local maximum along the gradient direction.  
    1. Bin each angle into one of the 4 possible directions (0°, 45°, 90°, 135°).  
    To do this we split the directions into sections:  
    0°      22.5°   67.5°   112.5°   157.5°   180°  
    |-----|-----|-----|-----|-----|  
    0°      45°    90°    135°    0°  
    2. Keep the pixel only if it's the local maximum along the edge.  
    3. Return the image with thinned edges.  
    """  
    img_h, img_w = sobel_edges.shape  
    img_thin_edges = np.zeros_like(sobel_edges, dtype=np.float64)  
  
    for i in range(1, img_h - 1):  
        for j in range(1, img_w - 1):  
            angle = angles[i, j]  
  
            if (angle < 22.5) or (angle >= 157.5):  
                ...  
            elif (angle >= 22.5) and (angle < 67.5):  
                ...  
            elif (angle >= 67.5) and (angle < 112.5):  
                ...  
            else:  
                ...  
  
            # Keep pixel only if it's the local maximum  
            if sobel_edges[i, j] >= n1 and sobel_edges[i, j] >= n2:  
                img_thin_edges[i, j] = sobel_edges[i, j]  
            else:  
                img_thin_edges[i, j] = 0  
  
    return img_thin_edges.astype(np.uint8)
```

Double Thresholding

Ogni pixel viene **classificato** in base al suo magnitudo G rispetto a una **soglia alta** u e una **soglia bassa** l scelte dall'utente:

- **Edge forte:** $G > h$
- **Edge debole:** $l < G < h$
- **Edge falso:** $G < l$

Le soglie vengono calcolate come percentuale del valore massimo di G per adattarsi ad ogni immagine

```
def double_threshold(thinned_edges, low_ratio=0.1, high_ratio=0.3):
    # Get thresholds by multiplying the maximum magnitude by the ratio.
    # This way, each threshold is adapted to each image.
    low_thr = thinned_edges.max() * low_ratio
    high_thr = thinned_edges.max() * high_ratio

    strong_pix = 200
    weak_pix = 100
    img_dthr = np.zeros_like(thinned_edges, dtype=np.float64)

    # Apply thresholds to each edge by using numpy's boolean indexing.
    # thinned_edges >= high creates a boolean array whose
    # values = 1 when thinned_edges >= high.
    # The weak mask combines two boolean arrays via &.
    # Values in both arrays must be = 1, else it is set as = 0.
    strong_mask = thinned_edges >= high_thr
    weak_mask = (thinned_edges >= low_thr) & (thinned_edges < high_thr)

    img_dthr[strong_mask] = strong_pix
    img_dthr[weak_mask] = weak_pix

    return img_dthr.astype(np.uint8), strong_pix, weak_pix
```

Isteresi

Gli edge deboli connessi ad un edge forte vengono **promossi** mentre gli edge deboli isolati vengono **scartati**.

La propagazione avviene tramite **stack**:

- Inserire tutti edge forti nello stack
- Controllare gli 8 vicini per ognuno
- Se vicino debole → promuovere ed aggiungere allo stack
- Ripetere fino ad esaurimento



```
def hysteresis(img_dthr, strong_pix, weak_pix):
    img_h, img_w = img_dthr.shape
    img_final_edges = img_dthr.copy()

    # Find positions of all strong pixels.
    strong_rows, strong_cols = np.where(img_final_edges == strong_pix)
    # Pair them into (row, col) tuples.
    strong_pos = zip(strong_rows, strong_cols)
    # Convert to list to append and pop.
    stack = list(strong_pos)

    while stack:
        i, j = stack.pop()

        # Check the 8 neighbors: di - row offset, dj - column offset.
        for di in [-1, 0, 1]:
            for dj in [-1, 0, 1]:
                # Skip the pixel itself.
                if di == 0 and dj == 0: continue
                # Get the neighbor's position.
                ni = i + di
                nj = j + dj

                # Check the bounds before accessing.
                if 0 <= ni < img_h and 0 <= nj < img_w:
                    # Promote weak edge to strong
                    if img_final_edges[ni, nj] == weak_pix:
                        img_final_edges[ni, nj] = strong_pix
                        stack.append((ni, nj))

    # Discard remaining weak edges
    img_final_edges[img_final_edges == weak_pix] = 0

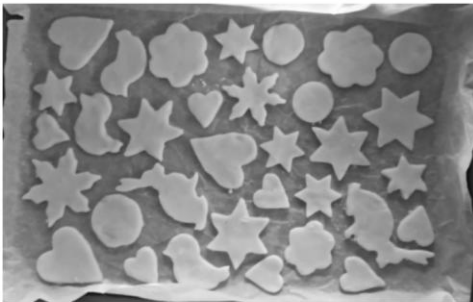
    return img_final_edges
```

Risultati - Caso Semplice

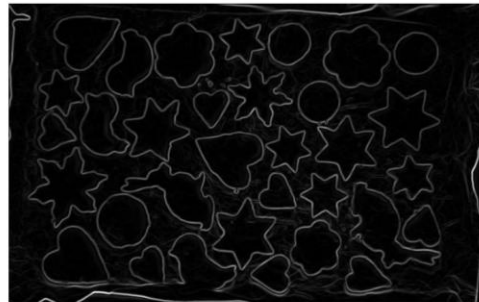
Original



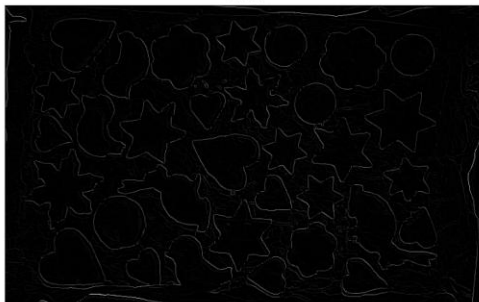
Blurred



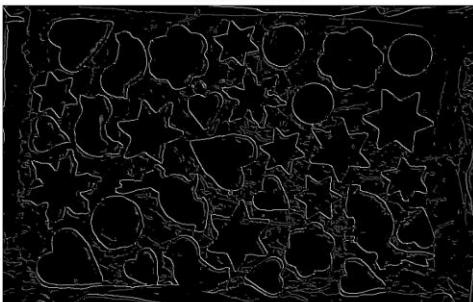
Sobel



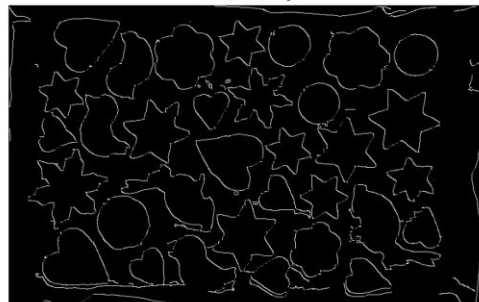
NMS



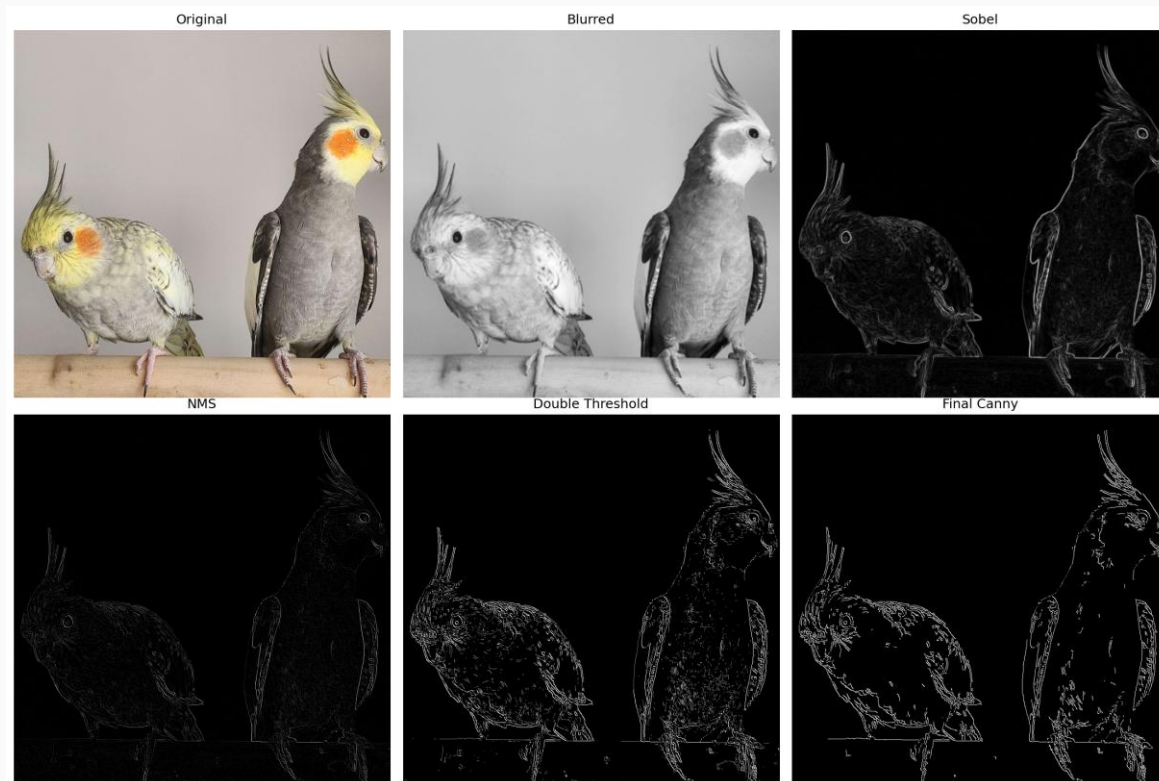
Double Threshold



Final Canny



Risultati - Caso Reale



Risultati – Caso Difficile



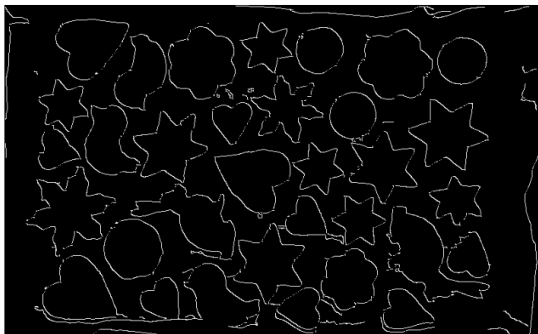
$$k = 5 \times 5, \sigma = 1.0, l = 0.1, u = 0.3$$



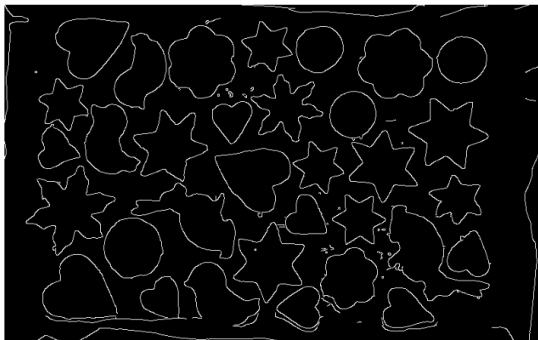
$$k = 5 \times 5, \sigma = 2.0, l = 0.2, u = 0.3$$

Confronti – OpenCV

My Canny implementation



OpenCV Canny implementation



1

Similarità

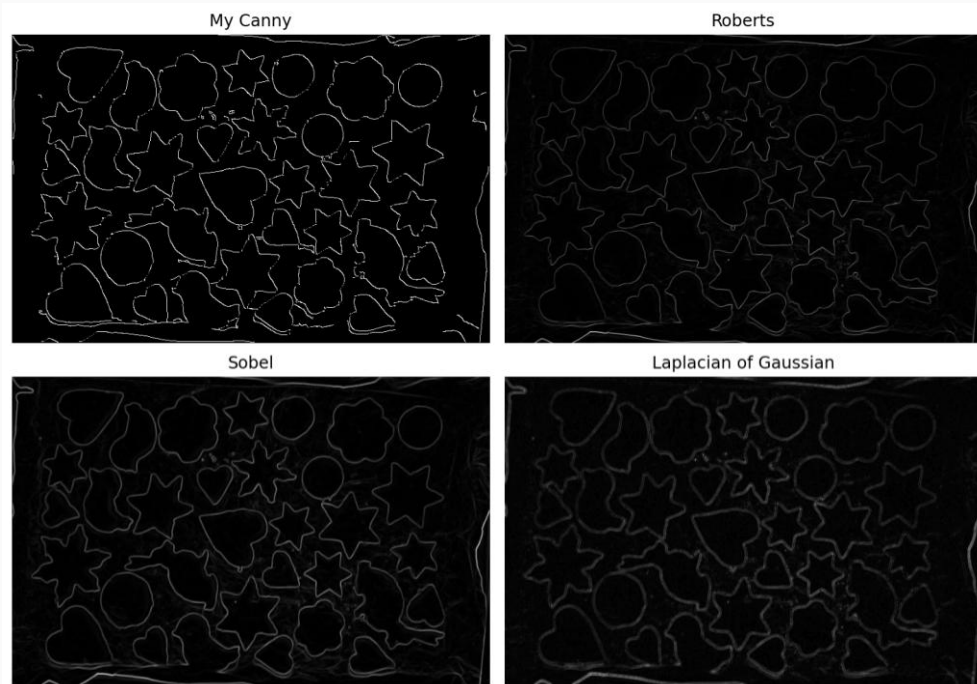
- Stessi edge principali
- Edge **sottili**
- Edge **ben localizzati**

2

Differenze

- OpenCV produce **contorni più completi**
- Rumore della carta forno **riprodotto diversamente**

Confronti – Altri Detector



1

Roberts

Abbastanza **rumore di fondo**
per assenza di smoothing

2

Sobel

Edge più definiti ma **nessun
assottigliamento**

3

LoG

Edge più sottili grazie ai zero-
crossing, ma **effetti di sdoppiamento**

4

Canny

Combina i tre criteri fondamentali





Thanks!



Do you have any questions?

Adrienne Macazar

Fondamenti dell'Intelligenza Artificiale 2025-26

Università di Parma

adriennechloerayos.maczar@studenti.unipr.it



CREDITS: This presentation template was created by [Slidesgo](#), and
includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

Please keep this slide for attribution
